95.7
Pass

Bharath Krishna N R

PDF generated at: 12 Feb 2026 13:32:22 UTC

View this report on HackerRank

Score

95.7% • 167.5 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 36 min 40 sec on 12 Feb 2026 18:17:28 IST

Candidate Information

Email	bharathkrish576@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	12 Feb 2026 18:17:28 IST
Time taken	36 min 40 sec/ 90 min
Work Experience	5 years
Invited by	Nancy
Invited on	11 Feb 2026 19:50:41 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	50%

2	Spring Boot Intermediate	83%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	70%	Hard	50%
Spring	50%	JPA	50%
Spring Data	50%	Transactional	50%
Medium	98%	Sorting	100%
Greedy	100%	Arrays	100%
Two Pointers	100%	Java	100%
Multithreading	100%	Interfaces	100%
OOP	100%		

Questions

Spring Boot (Advanced) • 17.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status

1	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	24 sec	✓
2	Determine the Output Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 1 sec	✗
3	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	2.5/5	-	1 min 36 sec	✓
4	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	55 sec	✓
5	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	19 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Capable Models Coding	Problem Solving (Intermediate)	75/75	-	18 min 33 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	13 min 26 sec	✓

Candidate Feedback

Good User Experience

1. Specification

✓ Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● **Specification** ✓
- ○ Criteria
- ○ Query

No comments.

2. Determine the Output

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
   public static void main(String[] args) {
      SpringApplication.run(SpringQuestionsApplication.class, args);
   }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Truth is behind the lies");
      System.out.println("This is working perfectly");
      throw new Exception("Does this work?");
   }
}
```

Truth.java

```
package com.hackerrank.spring.reality;
```

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"

Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions

Explanation: `Lie` never runs, so its prints and exception do not occur.

Option [C] – Prints from Lie only

Explanation: `Truth` is discovered and executed; `Lie` is not.

Option [D] – Prints from Lie, then Truth

Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Truth is in your eyes ✓

- Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- **Truth is behind the lies This is working perfectly And exception occurs and application stops**
- Truth is behind the lies This is working perfectly And exception occurs containing “Does this work?” Truth is in your eyes

No comments.

3. Spring Data Transactional

Partially correct

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
    @Id
    private Long id;
    private String name;
    private Integer courseLength;
    private Integer courseLevel;
    @JsonBackReference
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id")
    private Student student;
```

```
    private boolean inProgress;
    //getter & setters
}
```

Student.java

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
    private String surname;
    @JsonManagedReference
    @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
    private List<Lesson> lessons;
    //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods

    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);

        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
```

```
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] It will try to delete the student then the DataIntegrityViolationException will be thrown.
- [ ] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".
- [x] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.
- [ ] It will throw a RuntimeException with the message "Student not found".
- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

4. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Configuration class annotated with @EnableGlobalMethodSecurity
- ○ Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)
- ● **Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)** ✓
- ○ None above

No comments.

5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ private void manage(HttpSecurity http) {   http.csrf().allowed() }
- ○ private void manage(HttpSecurity http) {   http.csrf() }
- ○ private void manage(HttpSecurity http) {   http.csrf().enable() }
- ● ** None of the above. CSRF is configured by default when Spring Security Dependency is added. ** ✓

No comments.

6. Capable Models

Correct

Coding Medium Sorting Greedy Arrays Two Pointers

Question description

Given *n* machine learning models, each with an associated cost and feature compatibility:

- *cost[i]* represents the cost of the ith model
- *featureAvailability[i]* is a binary string indicating suitability for two distinct features:
  - "00": not equipped for either feature
  - "01": suitable for feature A but not feature B
  - "10": suitable for feature B but not feature A
  - "11": suitable for both features

A set of models is *k*-capable if the number of models suitable for feature A and the number suitable for feature B are both greater than or equal to *k*.

For each value of *k* from 1 to *n*, determine the minimum cost required to assemble a *k*-capable set of models. Return an array of *n* integers, where the ith integer represents the minimum cost for an *i*-capable set. If no *i*-capable set exists, the ith integer should be -1.

Example

n = 6

cost = [3, 6, 9, 1, 2, 5]

featureAvailability = ["10", "01", "11", "01", "11", "10"]

featureAvailability = [“10”, “01”, “11”, “01”, “11”, “10”]

k	Optimal Set	Feature 1 Compatible	Feature 2 Compatible	Cost
1	[5]	[5]	[5]	2
2	[1, 4, 5]	[1, 5]	[4, 5]	3 + 1 + 2 = 6
3	[1, 3, 4, 5]	[1, 3, 5]	[3, 4, 5]	3 + 9 + 1 + 2 = 15
4	[1, 2, 3, 4, 5, 6]	[1, 3, 5, 6]	[2, 3, 4, 5]	3 + 6 + 9 + 1 + 2 + 5 = 26

For $k \geq 5$, there will be no capable set.

Hence, the answer is [2, 6, 15, 26, -1, -1].

Function Description

Complete the function *getMinimumCost* in the editor with the following parameters:

int cost[n]: the cost of machine learning models

string featureAvailability[n]: the compatibility string of models indicating their suitability for two features

Returns

int[n]: the ith integer is the minimum cost of a set of *i*-capable models

Constraints

- $1 \le n \le 10^5$
- $1 \le cost[i] \le 10^4$
- It is guaranteed that *featureAvailability[i]* is a binary string of length 2.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of machine learning models.

Each of the next *n* lines contains an integer *cost[i]*.

The next line contains an integer, *n*, the number of models.

Each of the next *n* lines contains a string *featureAvailability[i]*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN        FUNCTION
-----        --------
4      →   cost[] size n = 4
5      →   cost = [5, 6, 10, 1]
6
10
1
4      →   featureAvailability[] size n = 4
10     →   featureAvailability = ["10", "01", "11", "00"]
01
11
00
```

Sample Output

```
10
21
-1
-1
```

Explanation

k	Optimal Set	Feature 1 Compatible	Feature 2 Compatible	Cost
1	[3]	[3]	[3]	10
2	[1, 2, 3]	[1, 3]	[2, 3]	5 + 6 + 10 = 21

For $k \geq 3$, there is no capable set.

Hence, the answer is [10, 21, -1, -1].

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN        FUNCTION
-----        --------
2      →   cost[] size n = 2
1      →   cost = [1, 1]
1
2      →   featureAvailability[] size n = 2
10     →   featureAvailability = ["10", "10"]
10
```

Sample Output

```
-1
-1
```

Explanation

There is no possible way of forming a *k-capable* set for all *k* from 1 to *n*.

Interviewer guidelines

▼ SOLUTION

Skills: Sorting, Two Pointers, Greedy Algorithms

To find the minimum cost for models such that the system is k-capable, the ones with "00" are unusable, while "01" and "10" have to be used together always and "11" can be used independently. This is because there should be k models supporting both features A and B.

Also, to find for each k from 1 to n, if the minimum cost of a k-capable system is known, find the next unused model(s) with minimum cost such that it supports both features A and B, and add the cost to find the minimum cost of (k + 1)-capable system.

So, the target is to create an array in increasing order such that each element is the cost of adding model(s) such that it supports both features.

Create three arrays: cost_both containing the models “11”, cost_one containing the models “01”, and cost_two containing the models “10”. Then, sort these arrays in an increasing order. Then, create an array cost_sum such that the ith element in cost_sum is cost_one[i] + cost_two[i], representing the cost to convert one “01” and one “10” into one “11” in sorted order. Then, append the two arrays cost_both and cost_sum and sort them to get the final array of costs.

Optimal Solution:

```
def getMinimumCost(cost, featureAvailability):
    n = len(cost)

    x1 = [x for x, y in zip(cost, featureAvailability) if y == "10"]
    x2 = [x for x, y in zip(cost, featureAvailability) if y == "01"]

    x = sorted(map(sum, zip(sorted(x1), sorted(x2))), reverse=True)
    y = sorted([x for x, y in zip(cost, featureAvailability) if y == "11"], reverse=True)

    ans = []
    px = py = val = 0
    while x or y:
        if not x or (x and y and x[-1] > y[-1]):
            val += y.pop()
        else:
            val += x.pop()
        ans.append(val)
    return ans + [-1] * (n - len(ans))
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n log n)

The sorting algorithm takes O(n log n) which is the highest order of operations performed. Therefore, the time complexity is O(n log n).

Space Complexity - O(n)

There are different auxiliary arrays used to store the costs. Therefore, the space complexity is O(n).

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMinimumCost' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY cost
     *  2. STRING_ARRAY featureAvailability
     */

    public static List<Integer> getMinimumCost(List<Integer> cost, List<String> featureAvailability) {
    // Write your code here
    int n=cost.size();
    List<Integer> onlyA=new ArrayList<>();
    List<Integer> onlyB=new ArrayList<>();
    List<Integer> both=new ArrayList<>();
    for(int i=0;i<n;i++){
        String feature=featureAvailability.get(i);
        int currentCost=cost.get(i);
        if("11".equals(feature)){
            both.add(currentCost);
        }else if("10".equals(feature)){
            onlyA.add(currentCost);
        }else if("01".equals(feature)){
```

```
            onlyB.add(currentCost);
        }
    }
    Collections.sort(onlyA);
    Collections.sort(onlyB);
    Collections.sort(both);
    long[] prefixA=new long[onlyA.size()+1];
    long[] prefixB=new long[onlyB.size()+1];
    long[] prefixBoth=new long[both.size()+1];
    for(int i=0;i<onlyA.size();i++){
        prefixA[i+1]=prefixA[i]+onlyA.get(i);
    }
     for(int i=0;i<onlyB.size();i++){
        prefixB[i+1]=prefixB[i]+onlyB.get(i);
    }
     for(int i=0;i<both.size();i++){
        prefixBoth[i+1]=prefixBoth[i]+both.get(i);
    }
    List<Integer> result=new ArrayList<>(n);
    for(int k=1;k<=n;k++){
        long minimumCost=Long.MAX_VALUE;
        int maxBothUsable=Math.min(k,both.size());
        for(int countBoth=0;countBoth<=maxBothUsable;countBoth++){
            int remaining=k-countBoth;
            if(remaining<=onlyA.size() && remaining<=onlyB.size()){
                long totalCost=prefixBoth[countBoth] + prefixA[remaining] + prefixB[remaining];
                if(totalCost<minimumCost){
                    minimumCost=totalCost;
                }
            }
        }
        result.add(minimumCost == Long.MAX_VALUE ? -1 : (int) minimumCost);
    }
    return result;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
```

```java

        int costCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> cost = IntStream.range(0, costCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int featureAvailabilityCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<String> featureAvailability = IntStream.range(0, featureAvailabilityCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .collect(toList());

        List<Integer> result = Result.getMinimumCost(cost, featureAvailability);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0545 sec	37.2 KB
TestCase 1	Easy	Sample	Success	1	0.0619 sec	37.2 KB
TestCase 2	Easy	Sample	Success	1	0.0543 sec	37.5 KB
TestCase 3	Easy	Hidden	Success	2	0.0492 sec	37.8 KB
TestCase 4	Easy	Hidden	Success	2	0.0508 sec	37.7 KB
TestCase 5	Medium	Hidden	Success	5	0.2546 sec	60.4 KB
TestCase 6	Medium	Hidden	Success	5	0.2662 sec	60.4 KB
TestCase 7	Medium	Hidden	Success	5	0.2675 sec	57.5 KB
TestCase 8	Medium	Hidden	Success	5	0.159 sec	57.3 KB
TestCase 9	Hard	Hidden	Success	8	0.1691 sec	66.3 KB
TestCase 10	Hard	Hidden	Success	8	1.4183 sec	76.5 KB
TestCase 11	Hard	Hidden	Success	8	3.0179 sec	74.8 KB

TestCase 12	Hard	Hidden	Success	8	0.433 sec	78.3 KB
TestCase 13	Hard	Hidden	Success	8	2.8657 sec	77.2 KB
TestCase 14	Hard	Hidden	Success	8	2.3323 sec	86.4 KB

No comments.

7. Java: Safe Cracking

Correct

Coding Medium Java Multithreading Interfaces OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.
2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number

3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- *1* ≤ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ *10*

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
   public void rob(IActions actions) throws InterruptedException, ExecutionException{
      ExecutorService threadPool = Executors.newFixedThreadPool(2);
      String passcodeOfDoor = actions.unlockTheDoor();
      Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
      Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));
```

```
        String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
        String safeDepositBoxPin = hackSecretPinTask.get();

        actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
        threadPool.shutdown();
    }
}
```

Candidate's Solution

Language used: **Java 21**

```
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}


class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{
        String passcodeOfDoor= actions.unlockTheDoor();
        ExecutorService executorService = Executors.newFixedThreadPool(2);
        Future<String> pinFuture= executorService.submit(()-> actions.hackSecretPin(passcodeOfDoor));
        Future<String> boxNumbFuture= executorService.submit( ()-> actions.figureOutSafeDepositBoxNumber(passcodeOfDoor));
        String safetyDepositBoxPin = pinFuture.get() ;
```

```
        String safetyDespositBoxNumber = boxNumbFuture.get();
        executorService.shutdown();
        actions.openSafeDepositLock(safetyDespositBoxNumber, safetyDepositBoxPin);
    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }
```

```java

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	2.0612 sec	37.8 KB
Testcase 1	Hard	Hidden	Success	74	2.0517 sec	38.2 KB

No comments.